

Guess Market - Exercise 2

Name: Dan Alushe

ID: 212472864

Email: danalushe15@gmail.com

GitHub: <https://github.com/danal15/guess-market>

Bonuses implemented

I implemented all 4 bonuses. Bonuses 1 and 2 start switched off, the controls for them are in the top bar.

- Skins - 3 color schemes (Light, Dark, High contrast) in the "Skin" box. Each one changes the background, the buttons and the font and size of the labels. Starts on Light.
- Animations - 3 animations (a fade of the event details panel, a slide of the user details panel, a pulse on the file name after a load). All under 1 second, and off until the "Animations" box is checked.
- Charts - every event has a chart of the two option prices per trade (LMSR and order book), and every user has a chart of their balance over time.
- Create new event - the "Create new event..." button under the users table. The selected user fills in the details for the chosen method and becomes the MM of the new event.

The save & load from exercise 1 is also still there ("Save Progress" button, .gmstate file). It is not one of the bonuses of this exercise, I just kept it.

How to run

- Put engine.jar, guess-market.jar and run.bat in the same folder and double click run.bat (or run: `java -jar guess-market.jar`).
- JavaFX is packed inside guess-market.jar, so nothing needs to be installed except Java 25.
- Load a file with the "Load File" button (file chooser). Only exercise 2 files are supported. The same button also opens a .gmstate file saved with "Save Progress".
- No sample files are included, the files from the course site load as they are.

How to use it

- Events tab - view only. All the events with filters by method / status / commission (each one has "All"). Clicking an event shows its details: for LMSR the prices, account, trade history and chart; for an order book both books, the statistics, the participants and the chart.
- Users tab - this is where the actions are. Pick a user, then pick an event, and the buttons for what that user can do right now appear: open / close for the MM, buy shares or place an order when the event is active. A button that is not allowed is disabled with a tooltip that says why.
- Money is always shown with \$ so it is not confused with a number of shares. Number columns sort by value.

Project structure

Two modules like before: engine (the system, no JavaFX in it) and desktop (the UI, JavaFX + FXML + CSS). The UI only talks to the GMEngine interface and gets DTO objects back, it never touches the engine's internal objects.

Main new classes: Event is now abstract, with LmsrEvent and OrderBookEvent extending it. OrderMatcher does the order book logic - first matches against the resting orders, then tries to mint against the other option, then rests what is left. OrderBook / Order / ExecutedTrade are the book of one option, one order and one trade. User / Holding are a user with a balance and what they hold in each event. Market holds everything (this is the object that gets serialized). MarketFileLoader reads the XML and does all the checks. In desktop: RootController,

EventsTabController and UsersTabController for the three screens, EventDetailView and TradeForms for the parts both tabs share, CreateEventDialog, SkinManager and AnimationManager for the bonuses.

Decisions I made

- All the actions are in the Users tab, the Events tab is only for looking (the exercise says participation is done from the users area).
- When two orders meet at different prices, the one already in the book gets its own price and the new one gets the better deal. This is what the simulation file does.
- Mint happens when the two prices together reach d , and only after a normal match was tried first (a normal match is cheaper for the buyer).
- Money for a buy order is taken only when it is filled, not when it rests in the book. This is how a user can go below zero - they are told about it and from then on they are blocked, like the exercise says.
- A user cannot sell shares they do not have.
- Before confirming a buy or an order the dialog shows what it will cost and warns if the balance is not enough. The engine does the same check, so they always agree.
- A blocked user cannot do anything, including closing their own event.
- Prices are whole cents, from 0.01 up to d minus 0.01.
- A purchase must cost at least \$0.01. In LMSR an option that was pushed very low can cost a fraction of a cent, which would give shares for nothing. This is refused with a message. A cheap option at \$0.02 can still be bought normally.
- The value of a holding uses the last trade price, or the mid price if there was no trade yet, or $d/2$ if the book is empty.
- If the initial investment of an order book event does not divide by d , only whole pairs of shares are created and the MM pays only for them.
- Commissions go to the MM's own account, both on-purchase and on-close.
- When an LMSR event closes, what is left in the event account after paying the winners goes back to the MM, so the account ends at 0.
- The XML is read with the parser that comes with Java, no extra libraries.
- The console UI from exercise 1 was removed, the engine was reused and extended.